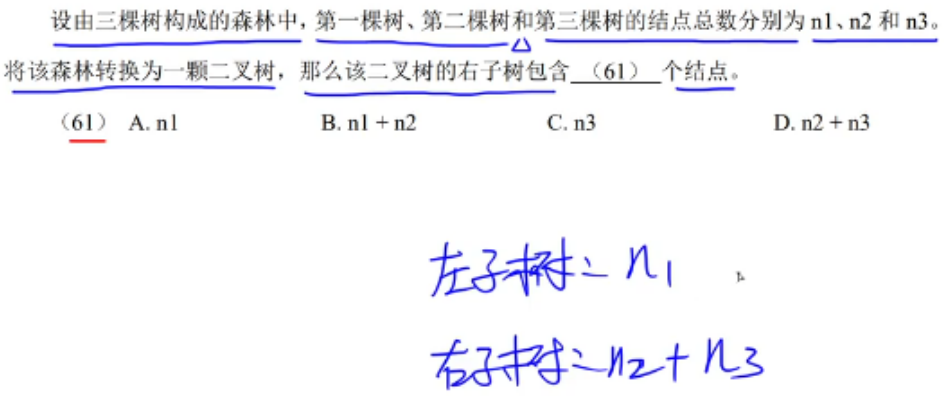
java

- 类图中的符号
  - `+` public
  - `-` private
  - `#` protected
  - `~` package

N黄后问题： N：6时，共4种((5 3 1 6 4 2) (4 1 5 2 6 3) (3 6 2 5 1 4) (2 4 6 1 3 5))

邻接表o(n+e), 邻接矩阵 o(n^2)



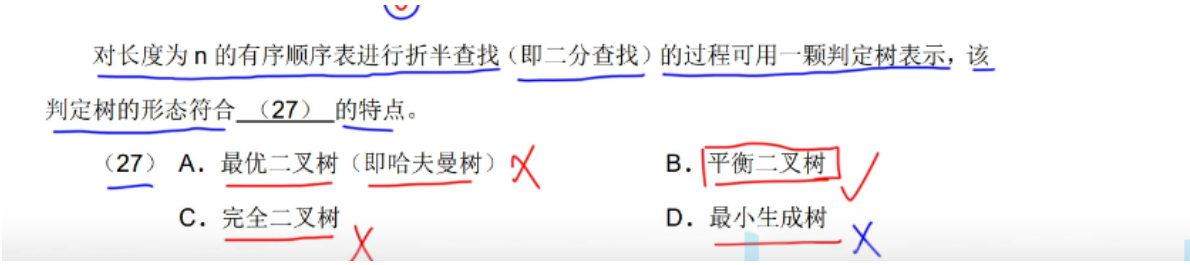
CPU与IO采用异步传输

SFTP文件 安全传输

ACL可以防止未授权的人访问内部网络

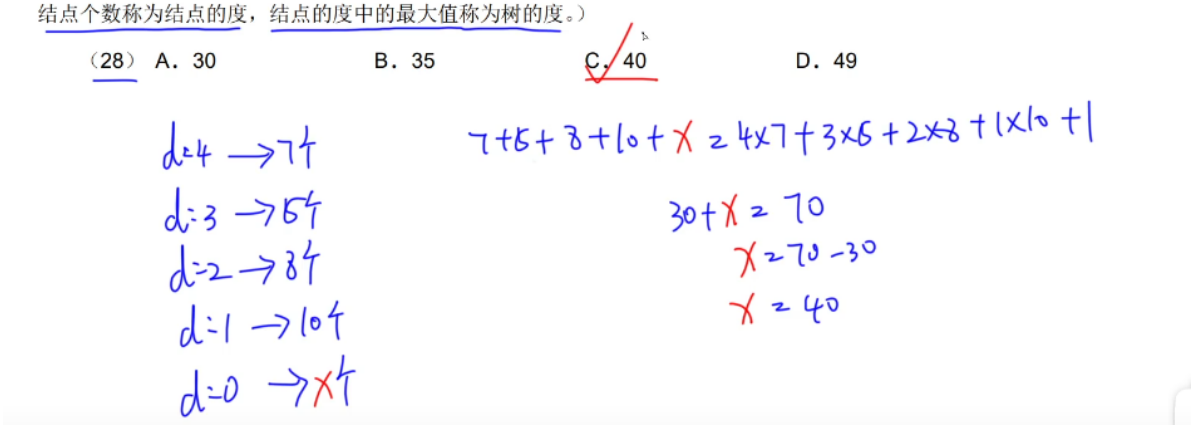
面向对象程序设计语言为面向对象 实现

用线性探测法解决冲突容易产生聚集问题



已知树 T 的度为 4，且度为 4 的结点数为 7 个、度为 3 的结点数为 5 个、度为 2 的结点数为 8 个、度为 1 的结点数为 10 个，那么树 T 的叶子结点个数为（28）。（注：树中结点个数称为结点的度，结点的度中的最大值称为树的度。）

（28） A．30　　B．35　　C．40　　D．49

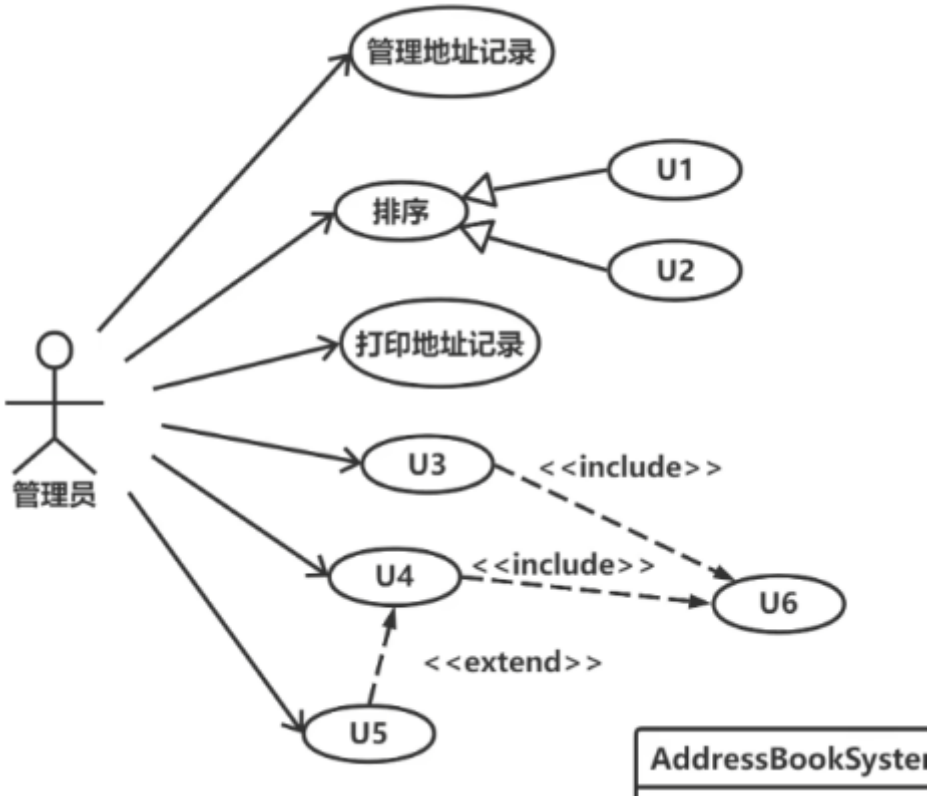


- 用例 关系

包含（include）： include为包含关系，当两个或多个用例中共用一组相同的动作，
这时可以将这组相同的动作抽出来作为一个独立的子用例，
供多个基用例所共享。因为子用例被抽出，基用例并非一个完整的用例，
所以include关系中的基用例必须和子用例一起使用才够完整，
子用例也必然被执行。include关系在用例图中使用带箭头的虚线表示
（在线上标注<<include>>），箭头从基用例指向子用例。

扩展（extend）： extend关系是对基用例的扩展，基用例是一个完整的用例，
即使没有子用例的参与，也可以完成一个完整的功能。extend的基用例中将存在一个扩展点，
只有当扩展点被激活时，子用例才会被执行。 extend关系在用例图中使用带箭头的虚线表示
（在线上标注<extend>），箭头从子用例指向基用例。

泛化（generalization）： 泛化关系是一种继承关系，子用例将继承基用例的所有行为，
关系和通信关系，也就是说在任何使用基用例的地方都可以用子用例来代替。
泛化关系在用例图中使用空心的箭头表示，箭头方向从子用例指向基用例。



父图与子图平衡

- 父图中的加工 输入、输出、数据流必须与子图的输入、输出、数据流方向、数量、名字上相同

父图与子图如何保持平衡

- 首先父图的基础数据流与子图的基础数据流方向、数量、名字上相同
- 其次父图中的一条数据流可以对应子图的若干条数据流，只要子图上的数据流能够对应完整即可

【问题 4】（3 分）

根据说明，采用结构化语言对“订单处理”的加工逻辑进行描述。

（4）订单处理。收到订餐请求后，向外卖平台请求配送。外卖平台接到请求后发布配送单，由平台骑手接单，外卖平台根据是否有骑手接单返回接单状态。若外卖平台接单成功，系统给支付系统发送支付请求，接收支付状态。支付成功，更新订单状态为已接单，向商户发送订餐请求并由商户打印订单，给用户发送订单状态；若支付失败，更新订单状态为下单失败，向外卖平台请求取消配送，向用户发送下单失败。若系统接到外卖平台返回接单失败或超时未返回接单状态，则更新订单状态为下单失败，向用户发送下单失败。

```
收到订餐请求后，向外卖平台请求配送。
外卖平台接到请求后发布配送单，由平台骑手接单，
外卖平台根据是否有骑手接单返回接单状态。

IF 外卖平台接单成功 THEN
    系统给支付系统发送支付请求，接收支付状态。
    IF 支付成功 THEN
        更新订单状态为已接单，
        向商户发送订餐请求并由商户打印订单，
        给用户发送订单状态。
    ELSE IF 支付失败 THEN
        更新订单状态为下单失败，
        向外卖平台请求取消配送，
        向用户发送下单失败。
    ENDIF
ELSE IF 系统接到外卖平台返回接单失败或超时未返回接单状态 THEN
    更新订单状态为下单失败，
    向用户发送下单失败。
ENDIF
```

- 0-1背包、矩阵边乘，　动态规画算法
- N皇后问题　回溯法
-

```
interface Animal {
   public void eat();
   public void travel(int a);
}

public class MammalInt implements Animal{

   public void eat(){

   }

   public void travel(int a){

   }
}
```

```
abstract class Animal {
    String name;

    public Animal(String name) {
        this.name = name;
    }

    // 抽象类有函数体，肯定不是abstract
    public String getData(){
        return ...
    }

    public abstract void sound();


}

class Dog extends Animal {
    public Dog(String name) {
        super(name);
    }


    void sound() {
        System.out.println(name + " is barking.");
    }
}
```

在上述代码中，`Animal`是一个抽象类，它包含一个抽象方法`sound()`和一个非抽象方法`sleep()`。`Dog`是`Animal`的子类，必须实现抽象方法`sound()`。

如果Dog类有抽象类方法，那Animal里的方法肯定是abstract

```
class Product{
    public void setA(String s){}
    public void setB(String s){}
}

public Build{
    private Product product;
    ...
    product.setA(s);
}
```

一个类中直接调另外的内，拿过来定义后用

```
class Work implements 接口{
    private String name;
    private String age;
    private WorkE work;

    public Object Clone(){
        Work obj = new Work(); // 复制时，取父类实例化
        obj.name = this.name;
        obj.age = this.age;
    }

    public Resume(WorkE work){
        // 当参数是一个对象时，赋值需要强制转换，并使用.Clone()复制
        this.work = (WorkE) work.Clone()
    }
}
```

例题

13 下试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入 （n） 处的字句写在答题纸的对应栏内。

【说明】

欲开发一个绘图软件，要求使用不同的绘图程序绘制不同的图形，以绘制直线和圆形为例，对应的绘图程序如表 6-1 所示。

表 6-1 不同的绘图程序

	DP1	DP2
绘制直线	draw_a_line(x1,y1,x2,y2)	drawline(x1,y1,x2,y2)
绘制圆	draw_a_circle(x, y, r)	drawcircle(x, y, r)

该绘图软件的扩展性要求，将不断扩充新的图形和新的绘图程序。为了避免出现类爆炸的情况，现采用桥接（Bridge）模式来实现上述要求，得到如图 6-1 所示的类图。

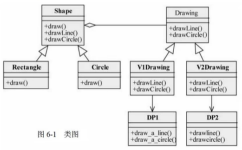


图 6-1 类图

【Java 代码】

```
(1) Drawing {
    (2) ;
    (3) ;
}

class DP1 {
    static public void draw_a_line(double x1, double y1, double x2, double y2
    { /* 代码省略 */ }
    static public void draw_a_circle(double x, double y, double r)
    { /* 代码省略 */ }
}

class DP2 {
    static public void drawline(double x1, double y1, double x2 ,double y2)
    { /* 代码省略 */ }
    static public void drawcircle (double x, double y, double r)
    { /* 代码省略 */ }
}
```

```
class V1Drawing implements Drawing {
    public void drawLine(double x1, double y1, double x2 ,double y2)
    { /* 代码省略 */ }
    public void drawCircle(double x, double y, double r) { (4) ; }
}
```

```
class V2Drawing implements Drawing {
    public void drawLine(double x1, double y1, double x2 ,double y2)
    { /* 代码省略 */ }
    public void drawCircle(double x, double y, double r) { (5) ; }
}
```

```
abstract class Shape {
    private Drawing _dp;
    (6) ;
    Shape(Drawing dp) { _dp = dp; }
    public void drawLine(double x1, double y1, double x2 ,double y2) { _dp.drawLine(x1, y1, x2, y2)
    public void drawCircle (double x, double y, double r) { _dp.drawCircle(x, y, r); }
}
```

```
class Rectangle extends Shape {
    private double _x1, _x2, _y1, _y2;
    public Rectangle(Drawing dp, double x1, double y1, double x2, double y2)
    { /* 代码省略 */ }
    public void draw() { /* 代码省略 */ }
}

class Circle extends Shape {
    private double _x, _y, _r;
    public Circle(Drawing dp, double x, double y, double r)
    { /* 代码省略 */ }
    public void draw() { drawCircle(_x, _y, _r); }
}
```

参考答案

（1）interface
（2）void drawLine(double x1, double y1, double x2, double y2)
（3）void drawCircle(double x, double y, double r)
（4）DP1.draw_a_circle(x, y, r)
（5）DP2.drawcircle(x, y, r)
（6）abstract public void draw()

12 下试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入 （n） 处的字句写在答题纸的对应栏内。

【说明】

现欲开发一个软件系统，要求能够同时支持多种不同的数据库，为此采用抽象工厂模式设计该系统。以 SQL Server 和 Access 两种数据库以及系统中的数据库表 Department 为例，其类图如图 6-1 所示。

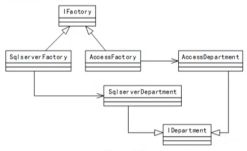


图 6-1 类图

【Java 代码】

```
import java.util.*;


class Department { /*代码省略*/ }


interface IDepartment {
    (1) ;
    (2) ;
}

class SqlserverDepartment (3) {
    public void Insert(Department department) {
        System.out.println("Insert a record into Department in SQL Server!")
        // 其余代码省略
    }


    public Department GetDepartment(int id) {
        /*代码省略*/
    }
}
```

```
class AccessDepartment (4) {
    public void Insert(Department department) {
        System.out.println("Insert a record into Department in ACCESS!");
        // 其余代码省略
    }


    public Department GetDepartment(int id) {
        /* 代码省略 */
    }
}


(5) {
    (6) ;
}
```

参考答案

（1）void Insert(Department department)
（2）Department GetDepartment(int id)
（3）implements IDepartment
（4）implements IDepartment
（5）interface IFactory
（6）IDepartment CreateDepartment()

17 上试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入 （n） 处的字句写在答题纸的对应栏内。

【说明】

某快餐厅主要制作并出售儿童套餐，一般包括主餐（各类比萨）、饮料和玩具，其餐品种类可能不同，但其制作过程相同。前台服务员（Waiter）调度厨师制作套餐。现采用生成器（Builder）模式实现制作过程，得到如图 6-1 所示的类图。

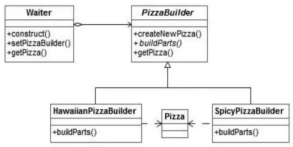


图 6-1 类图

【Java 代码】

```
class Pizza {
    private String parts;
    public void setParts(String parts) { this.parts = parts; }
    public String toString() { return this.parts; }
}


abstract class PizzaBuilder {
    protected Pizza pizza;
    public Pizza getPizza() { return pizza; }
    public void createNewPizza() { pizza = new Pizza(); }
    public (1) ;
}


class HawaiianPizzaBuilder extends PizzaBuilder {
    public void buildParts(){
        pizza.setParts("cross + mild + ham&pineapple");
    }
}
```

```
class SpicyPizzaBuilder extends PizzaBuilder {
    public void buildParts() {
        pizza.setParts("panbaked + hot + pepperoni&salami");
    }
}


class Waiter {
    private PizzaBuilder pizzaBuilder;
    public void setPizzaBuilder(PizzaBuilder pizzaBuilder) {/*设置构建器*/
        (2) ;
    }
    public Pizza getPizza() { return pizzaBuilder.getPizza(); }
    public void construct() { /* 构建 */
        pizzaBuilder.createNewPizza();
        (3) ;
    }
}
```

```
class FastFoodOrdering {
    public static void mainSting[]args) {
        Waiter waiter = new Waiter();
        PizzaBuilder hawaiian_pizzabuilder = new HawaiianPizzaBuild
        (4) ;
        (5) ;
        System.out.println("pizza: " + waiter.getPizza());
    }
}
```

```
(1): abstract void buildParts()
(2): this.pizzaBuilder = pizzaBuilder
(3): pizzaBuilder.buildParts()
(4): waiter.setPizzaBuilder(hawaiian_pizzabuilder)
(5): waiter.construct()
```

18 上试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入 （n） 处的字句写在答题纸的对应栏内。

【说明】

生成器（Builder）模式的意图是将一个复杂对象的构建与它的表示分离，使得同样的构建过程可以创建不同的表示。图 6-1 所示为其类图。

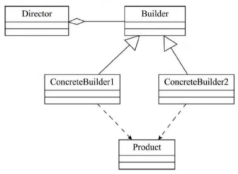


图 6-1　生成器模式类图

【Java 代码】

```
import java.util.*;

class Product {
    private String partA;
    private String partB;
    public Product() {}
    public void setPartA(String s) { partA = s;
    public void setPartB(String s) { partB = s;
}

interface Builder {
    public __(1)__;
    public void buildPartB();
    public __(2)__;
}
```

```
class ConcreteBuilder1 implements Builder {
    private Product product;
    public ConcreteBuilder1() { product = new Product(); }
    public void buildPartA() { __(3)__("Component A"); }
    public void buildPartB() { __(4)__("Component B"); }
    public Product getResult() { return product;}
}

class ConcreteBuilder2 implements Builder {
    // 代码省略
}
```

```
class Director {
    private Builder builder;
    public Director(Builder builder) { this.builder = builder; }
    public void construct() {
        __(5)__;
        // 代码省略
    }
}

class Test {
    public static void main(String[] args) {
        Director director1 = new Director(new ConcreteBuilder1());
        director1.construct();
    }
}
```

试题六参考答案

（1）void buildPartA()
（2）Product getResult()
（3）product.setPartA
（4）product.setPartA
（5）builder.buildPartA()或　builder.buildPartB()

13 上试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入 （n） 处的字句写在答题纸的对应栏内。

【说明】

现要求实现一个能够自动生成求职简历的程序，简历的基本内容包括求职者的姓名、性别、年龄及工作经历。希望每份简历中的工作经历有所不同，并尽量减少程序中的重复代码。

现采用原型模式（Prototype）来实现上述要求，得到如图 6-1 所示的类图。

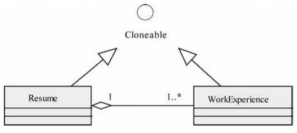


图 6-1　类图

【Java 代码】

```
class WorkExperience __(1)__ Cloneable { // 工作经历
    private String workDate;
    private String company;
    public Object Clone() {
        __(2)__;
        obj.workDate= this.workDate;
        obj.company = this.company;
        return obj;
    }
    // 其余代码省略
}
```

```
class Resume __(3)__ Cloneable { // 简历
    private String name;
    private String sex;
    private String age;
    private WorkExperience work;
    public Resume(String name) {
        this.name = name;
        work = new WorkExperience();
    }
    private Resume(WorkExperience work) {
        this.work = __(4)__;
    }
    public void SetPersonInfo(string sex, string age) { /* 代码略 */ }
    public void SetWorkExperience(string workDate, string company) {
        /* 代码略 */
    }
    public Object Clone() {
        Resume obj = __(5)__;
        // 其余代码省略
        return obj;
    }
}
```

参考答案

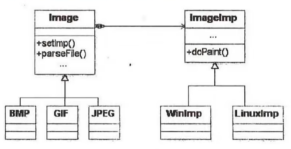
（1）implements
（2）WorkExperience obj = new WorkExperience()
（3）implements
（4）(WorkExperience)work.Clone()
（5）new Resume(this.work)
（6）(Resume)a.Clone()

09 上试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入 （n） 处的字句写在答题纸的对应栏内。

【说明】

现欲实现一个图像浏览系统，要求该系统能够显示 BMP、JPEG 和 GIF 三种格式的文件，并且能够在 Windows 和 Linux 两种操作系统上运行。系统首先将 BMP、JPEG 和 GIF 三种格式的文件解析为像素矩阵，然后将像素矩阵显示在屏幕上。系统需具有较好的扩展性以支持新的文件格式和操作系统，为满足上述需求并减少所需生成的子类数目，采用桥接（Bridge）设计模式进行设计所得类图如下图所示



类图

采用该设计模式的原因在于：系统解析 BMP、GIF 与 JPEG 文件的代码仅与文件格式相关，而在屏幕上显示像素矩阵的代码则仅与操作系统相关。

【Java 代码】

```
class Matrix { // 各种格式的文件最终都被转化为像素矩阵
    // 此处代码省略
}

abstract class ImageImp {
    public abstract void doPaint(Matrix m); // 显示像素矩阵 m
}

class WinImp extends ImageImp {
    public void doPaint(Matrix m) { /*调用 Windows 系统的绘制函数绘制像素矩阵 */
}

class LinuxImp extends ImageImp {
    public void doPaint(Matrix m) { /*调用 Linux 系统的绘制函数绘制像素矩阵 */
}
```

```
abstract class Image {
    public void setImp(ImageImp imp) {
        __(1)__ = imp;
    }
    public abstract void parseFile(String fileName);
    protected __(2)__ imp;
}

class BMP extends Image {
    public void parseFile(String fileName) {
    // 此处解析 BMP 文件并获得一个像素矩阵对象 m
        __(3)__; // 显示像素矩阵 m
    }
}

class GIF extends Image {
    // 此处代码省略
}
```

```
class JPEG extends Image {
    // 此处代码省略
}

public class javaMain {
    public static void main(String[] args) {
        // 在 windows 操作系统上查看 demo.bmp 图像文件
        Image image1 = __(4)__;
        ImageImp imageImp1 = __(5)__;
        __(6)__;
        Image1.parseFile("demo.bmp");
    }
}
```

参考答案

（1）this.imp　（2）ImageImp　（3）imp.doPaint(m)　（4）new BMP()
（5）new WinImp()　（6）image1.setImp(imageImp1)　（7）17

17 下试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入 （n） 处的字句写在答题纸的对应栏内。

【说明】

某图像预览程序要求能够查看 BMP、JPEG 和 GIF 三种格式的文件，且能够在 Windows 和 Linux 两种操作系统上运行。程序需具有较好的扩展性以支持新的文件格式和操作系统。为满足上述需求并减少所需生成的子类数目，现采用桥接（Bridge）模式进行设计，得到如图 6-1 所示的类图。

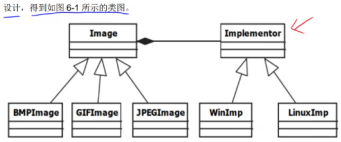


图 6-1 类图

【Java 代码】

```
import java.util.*;

class Matrix { // 各种格式的文件最终都被转化为像素矩
    // 此处代码省略
}

abstract class Implementor {
    public __(1)__; // 显示像素矩阵 m
}

class WinImp extends Implementor {
    public void doPaint(Matrix m) {
        // 调用 Windows 系统的绘制函数绘制像素矩阵
        // 此处代码省略
    }
}

class LinuxImp extends Implementor {
    public void doPaint(Matrix m) {
        // 调用 Linux 系统的绘制函数绘制像素矩阵
        // 此处代码省略
    }
}

abstract class Image {
    public void setImp(Implementor imp) { this.imp = imp; }
    public abstract void parseFile(String fileName);
    protected Implementor imp;
}

class BMPImage extends Image {
    // 此处代码省略
}

class GIFImage extends Image {
    public void parseFile(String fileName) {
        // 此处解析 BMP 文件并获得一个像素矩阵对象 m
        __(2)__; // 显示像素矩阵 m
    }
}

class JPEGImage extends Image {
    // 此处代码省略
}

class Main {
    public static void main(String[]args) {
        // 在 Linux 操作系统上查看 demo.gif 图像文件
        Image image = __(3)__;
        Implementor imageImp = __(4)__;
        __(5)__;
        Image.parseFile("demo.gif");
    }
}
```

试题六参考答案

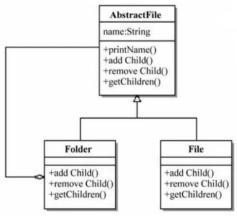
（1）abstract void doPaint(Matrix m)
（2）imp.doPaint(m)
（3）new GIFImage()
（4）new LinuxImp()
（5）image.setImp(imageImp)

09 下试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入 （n） 处的字句写在答题纸的对应栏内。

【说明】

现欲构造一文件/目录树，采用组合（Composite）设计模式来设计，得到的类图如下图所示：



【Java 代码】

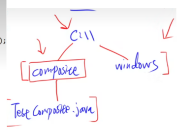
```
import java.util.ArrayList;
import java.util.List;

__(1)__ class AbstractFile {
    protected String name;
    public void printName() { System.out.println(name); }
    public abstract boolean addChild(AbstractFile file);
    public abstract boolean removeChild(AbstractFile file);
    public abstract List<AbstractFile> getChildren();
}

class File extends AbstractFile {
    public File(String name) { this.name = name; }
    public boolean addChild(AbstractFile file) { return false; }
    public boolean removeChild(AbstractFile file) { return false; }
    public List<AbstractFile> getChildren() { return __(2)__; }
}
```

```
class Folder extends AbstractFile {
    private List<AbstractFile> childList;
    public Folder(String name) {
        this.name = name;
        this.childList = new ArrayList<AbstractFile>();
    }
    public boolean addChild(AbstractFile file) {
        return childList.add(file);
    }
    public boolean removeChild(AbstractFile file) {
        return childList.remove(file);
    }
    public __(3)__ <AbstractFile> getChildren() { return __(4)__; }
```

```
public class Client {
    public static void main(String[] args) {
        // 创造一个树形的文件/目录结构
        AbstractFile rootFolder = new Folder("c:\\");
        AbstractFile compositeFolder = new Folder("composite");
        AbstractFile windowsFolder = new Folder("windows");
        AbstractFile file = new File("TestComposite.java");
        rootFolder.addChild(compositeFolder);
        rootFolder.addChild(windowsFolder);
        compositeFolder.addChild(file);
        // 打印目录文件数
        printTree(rootFolder);
    }
    private static void printTree(AbstractFile ifile) {
        ifile.printName();
        List<AbstractFile> children = ifile.getChildren();
        if (Children == null) return;
        for (AbstractFile file : Children) {
            __(5)__;
        }
    }
```



参考答案
（1）abstract （2）null （3）List
（4）childList （5）printTree(file)

10 下试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入 （n） 处的字句写在答题纸的对应栏内。

【说明】

某公司的组织结构图如图 6-1 所示，现采用组合（Composition）设计模式来构造该公司的组织结构，得到如图 6-2 所示的类图。

其中 Company 为抽象类，定义了在组织结构图上添加（Add）和删除（Delete）分公司/办事处或者部门的方法接口。类 ConcreteCompany 表示具体的分公司或者办事处，分公司或办事处下可以设置不同的部门。类 HRDepartment 和 FinanceDepartment 分别表示人力资源部和财务部。

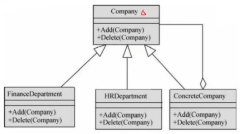


图 6-2 类图

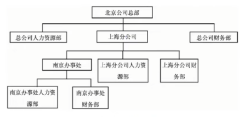


图 6-1 组织结构图

【Java 代码】

```
import java.util.*;

__(1)__ Company {
    protected String name;
    public Company(String name) { __(2)__ = name; }
    public abstract void Add(Company c); // 增加子公司、办事处或部门
    public abstract void Delete(Company c); // 删除子公司、办事处或部门
}
```

```
class ConcreteCompany extends Company {
    private List<__(3)__> children = new ArrayList<__(4)__>();
    // 存储子公司、办事处或部门
    public ConcreteCompany(String name) { super(name); }
    public void Add(Company c) { __(5)__.add(c); }
    public void Delete(Company c) { __(6)__.remove(c); }
}

class HRDepartment extends Company {
    public HRDepartment(String name) { super(name); }
    // 其他代码省略
}

class FinanceDepartment extends Company {
    public FinanceDepartment(String name) { super(name); }
    // 其他代码省略
}
```

```
public class Test {
    public static void main(String[] args) {
        ConcreteCompany root = new ConcreteCompany("北京总公司");
        root.Add(new HRDepartment("总公司人力资源部"));
        root.Add(new FinanceDepartment("总公司财务部"));

        ConcreteCompany comp = new ConcreteCompany("上海分公司");
        comp.Add(new HRDepartment("上海分公司人力资源部"));
        comp.Add(new FinanceDepartment("上海分公司财务部"));
        __(7)__;

        ConcreteCompany comp1 = new ConcreteCompany("南京办事处");
        comp1.Add(new HRDepartment("南京办事处人力资源部"));
        comp1.Add(new FinanceDepartment("南京办事处财务部"));
        __(8)__; // 其他代码省略
    }
}
```

参考答案
（1）abstract class
（2）this.name
（3）Company
（4）Company
（5）children
（6）children
（7）root.Add(comp)
（8）comp.Add(comp1)

21 上试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入 __(n)__ 处的字句写在答题纸的对应栏内。

【说明】

组合菜单是窗口风格的软件系统中经常采用的一种菜单风格形式，组合菜单（如图 6-1 示例）中包含的可选是一个菜单项（直接对应某个功能），也可能是一个子菜单。现在采用组合（composite）设计模式实现组合菜单，得到如图 6-2 所示的类图

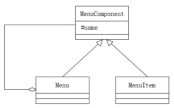


图 6-2 类图

【Java 代码】

```
import java.util.*;

abstract class MenuComponent {  // 构成组合菜单的元素
    __(1)__ String name;          // 菜单项或子菜单名称
    public void printName() { System.out.println(name); }
    public __(2)__ ;
    public abstract boolean removeMenuElement(MenuComponent element);
    public __(3)__ ;
}

class MenuItem extends MenuComponent {
    public MenuItem(String name) { this.name=name; }
    public boolean addMenuElement(MenuComponent element) { return false; }
    public boolean removeMenuElement(MenuComponent element) {
        return false;
    }
    public List<MenuComponent> getElement(){ return null; }
}
```

```
class Menu extends MenuComponent {
    private __(4)__ ;
    public Menu(String name) {
        this.name = name;
        this.elementList = new ArrayList<MenuComponent>();
    }
    public boolean addMenuElement(MenuComponent element) {
        return elementList.add(element);
    }
    public boolean removeMenuElement(MenuComponent element) {
        return elementList.remove(element);
    }
    public List<MenuComponent> getElement() { return elementList; }
}
```

```
class CompositeTest {
    public static void main(String[] args) {
        MenuComponent mainMenu = new Menu("Insert");
        MenuComponent subMenu = new Menu("Chart");
        MenuComponent element = new MenuItem("On This Sheet");
        __(1)__ ;
        subMenu.addMenuElement(element);
        printMenus(mainMenu);
    }

    private static void printMenus(MenuComponent ifile) {
        ifile.printName();
        List<MenuComponent> children = ifile.getElement();
        if (children == null) return;
        for(MenuComponent element : children) {
            printMenus(element);
        }
    }
}
```

(1)：protected
(2)：abstract boolean addMenuElement(MenuComponent element)
(3)：abstract List<MenuComponent> getElement()
(4)：List<MenuComponent> elementList
(5)：mainMenu.addElement(subMenu)

12 上试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入 __(n)__ 处的字句写在答题纸的对应栏内。

【说明】

某咖啡店当实咖啡时，可以根据顾客的要求在其中加入各种配料，咖啡店会根据所加入的配料来计算费用，咖啡店所供应的咖啡及配料的种类和价格如下表所示。

咖啡	价格/杯（¥）	配料	价格/份（¥）
蒸馏咖啡（Espresso）	25	摩卡（Mocha）	10
深度烘焙咖啡（DarkRoast）	20	搏泡（Whip）	8

现采用装饰器（Decorator）模式来实现计算费用的功能，得到如图 6-1 所示的类图

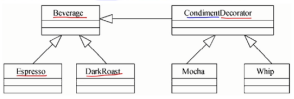


图 6-1 类图

【Java 代码】

```
import java.util.*;

__(1)__ class Beverage { // 饮料
    String description = "Unknown Beverage";
    public __(2)__ () { return description; }
    public __(3)__ ;
}

abstract class CondimentDecorator extends Beverage { // 配
    __(4)__ ;
}

class Espresso extends Beverage { // 蒸馏咖啡
    private final int ESPRESSO_PRICE = 25;
    public Espresso() { description = "Espresso"; }
    public int cost() { return ESPRESSO_PRICE; }
}
```

```
class DarkRoast extends Beverage { // 深度烘焙咖啡
    private final int DARKROAST_PRICE = 20;
    public DarkRoast() { description = "DarkRoast"; }
    public int cost() { return DARKROAST_PRICE; }
}
class Mocha extends CondimentDecorator { // 摩卡
    private final int MOCHA_PRICE = 10;
    public Mocha (Beverage beverage) { this.beverage = beverage; }
    public String getDescription() {
        return beverage.getDescription() + ", Mocha";
    }
    public int cost() { return MOCHA_PRICE + beverage.cost(); }
}
class Whip extends CondimentDecorator { // 奶泡
    private final int WHIP_PRICE = 8;
    public Whip (Beverage beverage) { this.beverage = beverage; }
    public String getDescription() {
        return beverage.getDescription() + ", Whip";
    }
    public int cost() { return WHIP_PRICE + beverage.cost(); }
}
```

```
public class Coffee {
    public static void main(String args[]) {
        Beverage beverage = new DarkRoast();
        beverage = new Mocha( __(5)__ );
        beverage = new Whip ( __(6)__ );
        System.out.println(beverage.getDescription() +"￥" + beverage.cost());
    }
}
```

参考答案

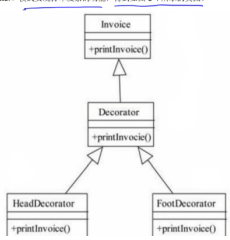
（1）abstract
（2）String getDescription
（3）abstract int cost()
（4）Beverage beverage
（5）beverage
（6）beverage

16 下试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入 __(n)__ 处的字句写在答题纸的对应栏内。

【说明】

某发票（Invoice）由抬头（Head）部分、正文部分和脚注（Foot）部分构成。现采用装饰（Decorator）模式实现打印发票的功能，得到如图 6-1 所示的类图。



此题复杂

【Java 代码】

```
class Invoice {
    public void printInvoice() {
        System.out.println("This is the content of the invoice!");
    }
}

class Decorator extends Invoice {
    protected Invoice ticket;
    public Decorator(Invoice t) {
        ticket = t;
    }
    public void printInvoice() {
        if (ticket != null)
            __(1)__ ;
    }
}
```

```
class HeadDecorator extends Decorator {
    public HeadDecorator(Invoice t) {
        super(t);
    }
    public void printInvoice () {
        System.out.println("This is the header of the invoice!");
        __(2)__ ;
    }
}

class FootDecorator extends Decorator {
    public FootDecorator(Invoice t) {
        super(t);
    }
    public void printInvoice() {
        __(3)__ ;
        System.out.println("This is the footnote of the invoice!");
    }
}
```

```
class Test {
    public static void main(String[] args) {
        Invoice t = new Invoice();
        Invoice ticket;
        ticket = __(4)__ ;
        ticket.printInvoice();
        System.out.println("--------------------");
        ticket = __(5)__ ;
        ticket.printInvoice();
    }
}
```

试题六参考答案

（1）ticket.printInvoice()
（2）super.printInvoice()
（3）super.printInvoice()
（4）new HeadDecorator(new FootDecorator(t))
（5）new HeadDecorator(new FootDecorator(null))

21 下试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入 __(n)__ 处的字句写在答题纸的对应栏内。

【说明】

一单元（PiyoPiyo）模式主要用于减少创建对象的数量，以降低内存占用，提高性能。先举例某一个网络游戏软件，它许多个玩家组队下棋，由于只有一台服务器，为下棋内存空间，采用享元模式实现该游戏，得到如图 6-1 所示的类图。

图6-1 类图

【Java 代码】

```
import java.util.*;

enum PieceColor { BLACK, WHITE } // 棋子颜色

class PiecePos { // 棋子位置
    private int x;
    private int y;
    public PiecePos(int a, int b) { x = a; y = b; }
    public int getX() { return x; }
    public int getY() { return y; }
}

abstract class Piece { // 棋子定义
    protected PieceColor m_color; // 颜色
    protected PiecePos m_pos; // 位置
    public Piece(PieceColor color, PiecePos pos) {
        m_color = color;
        m_pos = pos;
    }
    __(1)__ ;
}
```

```
class BlackPiece extends Piece {
    public BlackPiece(PieceColor color, PiecePos pos) {
        super(color, pos);
    }
    public void draw() { System.out.println("draw a blackpiece"); }
}
class WhitePiece extends Piece {
    public WhitePiece(PieceColor color, PiecePos pos) {
        super(color, pos);
    }
    public void draw() { System.out.println("draw a whitepiece"); }
}
```

```
class PieceBoard { // 棋盘上已有的棋子
    private static final ArrayList<2> m_arrayPiece = new ArrayList <> ();
    private String m_blackName; // 黑方名称
    private String m_whiteName; // 白方名称
    public PieceBoard(String black, String white) {
        m_blackName = black;
        m_whiteName = white;
    }
    // 一步棋，在棋盘上放一颗棋子
    public void SetPiece(PieceColor color, PiecePos pos) {
        __(2)__ piece = null;
        if (color == PieceColor.BLACK) { // 放黑子
            piece = new BlackPiece(color, pos); // 获取一颗黑子
            System.out.println(m_blackName + "在位置(" + pos.getX() + "," + pos.getY
            __(3)__ ;
        } else { // 放白子
            piece = new WhitePiece(color, pos); // 获取一颗白子
            System.out.println(m_whiteName + "在位置(" + pos.getX() + "," + pos.getY
            __(4)__ ;
        }
        m_arrayPiece.add(piece);
    }
}
```

(1)：public abstract void draw()
(2)：Piece piece
(3)：Piece
(4)：piece.draw()
(5)：piece.draw()

14 下试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入 （n） 处的字句写在答题纸的对应栏内。

【说明】

某灯具厂商欲生产一个灯具遥控器，该遥控器具有 7 个可编程的插槽，每个插槽都有开关按钮，对应着一个不同的灯。利用该遥控器能够统一控制房间中该厂商所有品牌灯具的开关，现采用 Command（命令）模式实现该遥控器的软件部分。Command 模式的类图如图 6-1 所示。

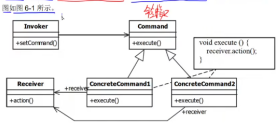


图 6-1 Command 模式类图

【Java 代码】

```
class Light {
    public Light() {}
    public Light(String name) { /* 代码省略 */ }
    public void on() { /* 代码省略 */ } // 开灯
    public void off() { /* 代码省略 */ } // 关灯
    // 其余代码省略
}

__(1)__ {
    public void execute();
}

class LightOnCommand implements Command { // 开灯命令
    Light light;
    public LightOnCommand(Light light) { this.light = light; }
    public void execute() { __(2)__; }
}

class LightOffCommand implements Command { // 关灯命令
    Light light;
    public LightOffCommand(Light light) { this.light = light; }
    public void execute() { __(3)__; }
}
```

```
class RemoteControl { // 遥控器
    Command[] onCommands = new Command[7]; // 开启槽
    Command[] offCommands = new Command[7]; // 关闭槽
    public RemoteControl() { /* 代码省略 */ }
    public void setCommand(int slot, Command onCommand, Command offCommand) {
        __(4)__ = onCommand;
        __(5)__ = offCommand;
    }
    public void onButtonWasPushed(int slot) {
    __(6)__;
    }
    public void offButtonWasPushed(int slot) {
    __(7)__;
    }
}
```

试题六参考答案

（1）interface Command

（2）light.on()

（3）light.off()

（4）onCommands[slot]

（5）offCommands[slot]

（6）onCommands[slot].execute()

（7）offCommands[slot].execute()